

```

validMoves = []
if blanky != SIZE - 1 and prevMove != DOWN:
    # Blank space is not on the bottom row.
    validMoves.append(UP)

if blankx != SIZE - 1 and prevMove != RIGHT:
    # Blank space is not on the right column.
    validMoves.append(LEFT)

if blanky != 0 and prevMove != UP:
    # Blank space is not on the top row.
    validMoves.append(DOWN)

if blankx != 0 and prevMove != LEFT:
    # Blank space is not on the left column.
    validMoves.append(RIGHT)

return validMoves

def getNewPuzzle():
    """Get a new puzzle by making random slides from the solved state."""
    board = getNewBoard()
    for i in range(DIFFICULTY):
        validMoves = getValidMoves(board)
        makeMove(board, random.choice(validMoves))
    return board

def solve(board, maxMoves):
    """Attempt to solve the puzzle in `board` in at most `maxMoves`
    moves. Returns True if solved, otherwise False."""
    print('Attempting to solve in at most', maxMoves, 'moves...')
    solutionMoves = [] # A list of UP, DOWN, LEFT, RIGHT values.
    solved = attemptMove(board, solutionMoves, maxMoves, None)

    if solved:
        displayBoard(board)
        for move in solutionMoves:
            print('Move', move)
            makeMove(board, move)
            print() # Print a newline.
            displayBoard(board)
            print() # Print a newline.

```